

Machine Learning Algorithms and Applications

Lecture 6

(Block 2)

Plan for Today



Cross-Validation



ExtraTrees



RandomForest

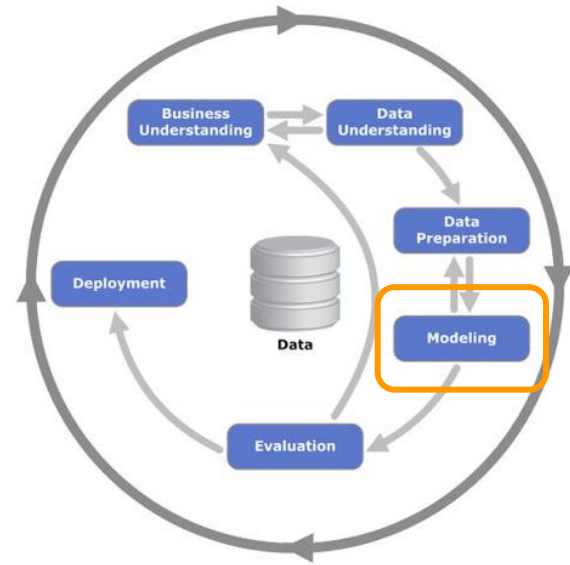


Hyperparameter Tuning

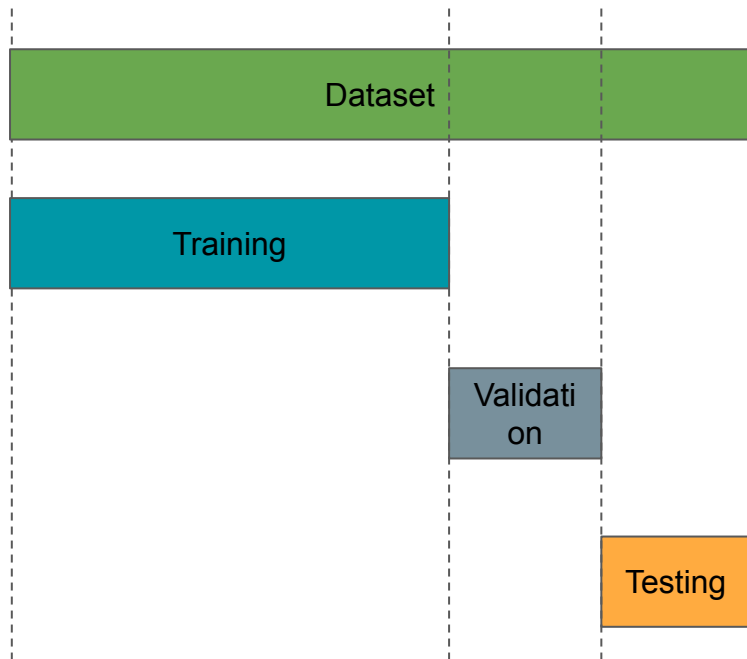


Lab

Cross-Validation



Splitting



Validation and testings sets are used for assessing model's ability to make correct predictions on unseen data.

Testing set should be used only a handful of times.

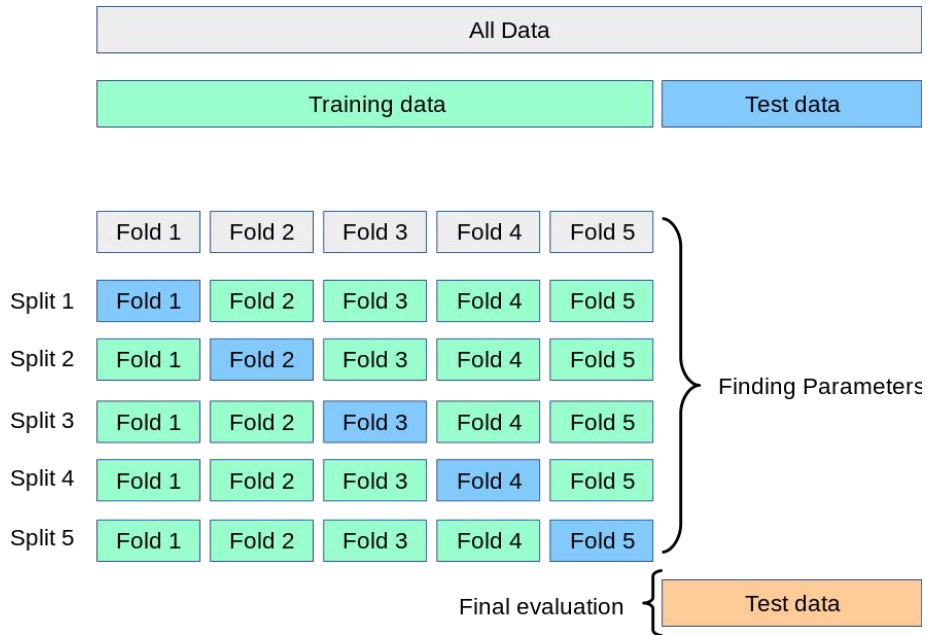
Validation is used for hyperparameter tuning.

But this means we are using only a subset of the training set. Also if we test a lot of combinations of hyperparameters then we will optimize for the validation set (won't be considered as unseen data anymore)

Cross-Validation Strategy

Create multiple samples of training and validation sets and train the models on these samples.

1. Shuffle the dataset randomly.
2. Split the dataset into k groups
3. For each unique group:
 - a. Take the group as a hold out or test data set
 - b. Take the remaining groups as a training data set
 - c. Fit a model on the training set and evaluate it on the test set
 - d. Retain the evaluation score and discard the model
4. Summarize the skill of the model using the sample of model evaluation scores



Sklearn Cross-Validation Score

```
from sklearn.svm import SVC
```

```
svc = SVC(C=1000.0, kernel='rbf', gamma=0.005)
```

```
from sklearn.model_selection import cross_val_score
```

```
cross_val_score(svc, X_train, y_train, cv=5, scoring='accuracy')
```

ML model with
hyperparameters

ML model with
hyperparameters

Number of splits
or cross validation
instance

Performance
metric

```
array([0.96..., 1. ..., 0.96..., 0.96..., 1. ...])
```

Sklearn k-fold

We can use k-fold directly to split the data

```
from sklearn.model_selection import KFold
```

```
kf = KFold(n_splits=2)
```

```
for train_index, test_index in kf.split(X):  
    X_train, y_train = X[train_index], y[train_index]  
    X_test, y_test = X[test_index], y[test_index]
```

Sklearn Stratified k-fold

Problem:

What if some of the target classes have less occurrence than others?
They may not be represented in CV splits

Solution:

Sample observations following the distribution of the target classes

```
from sklearn.model_selection import StratifiedKFold
```

```
skf = StratifiedKFold(n_splits=2)
```

```
for train_index, test_index in skf.split(X, y):  
    X_train, X_test = X[train_index], X[test_index]  
    y_train, y_test = y[train_index], y[test_index]
```


RandomForest

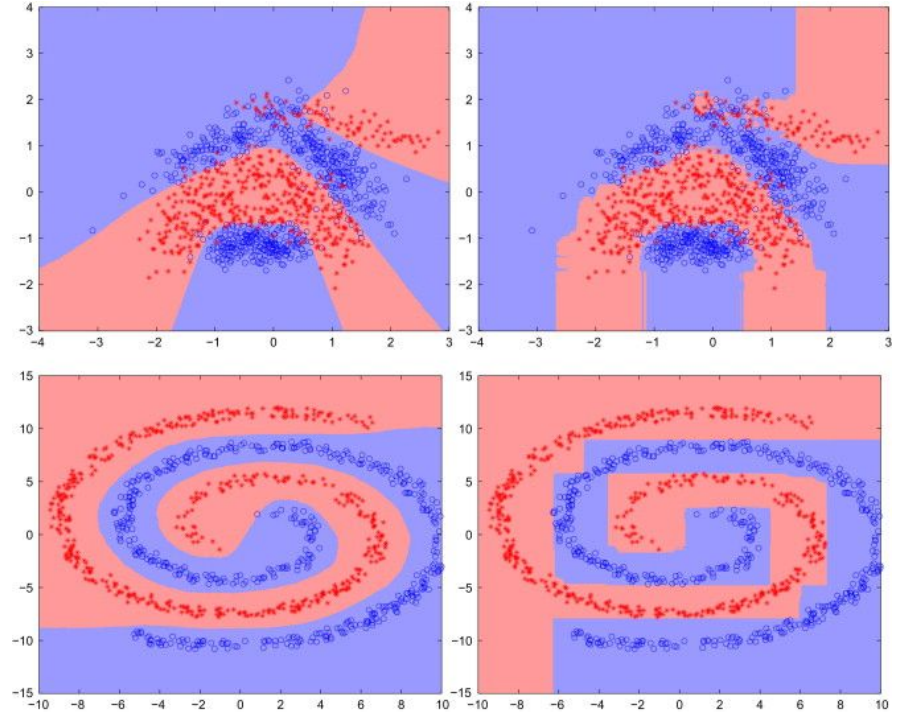


RandomForest

Ensemble algorithm using bagging

One of the most popular algorithms

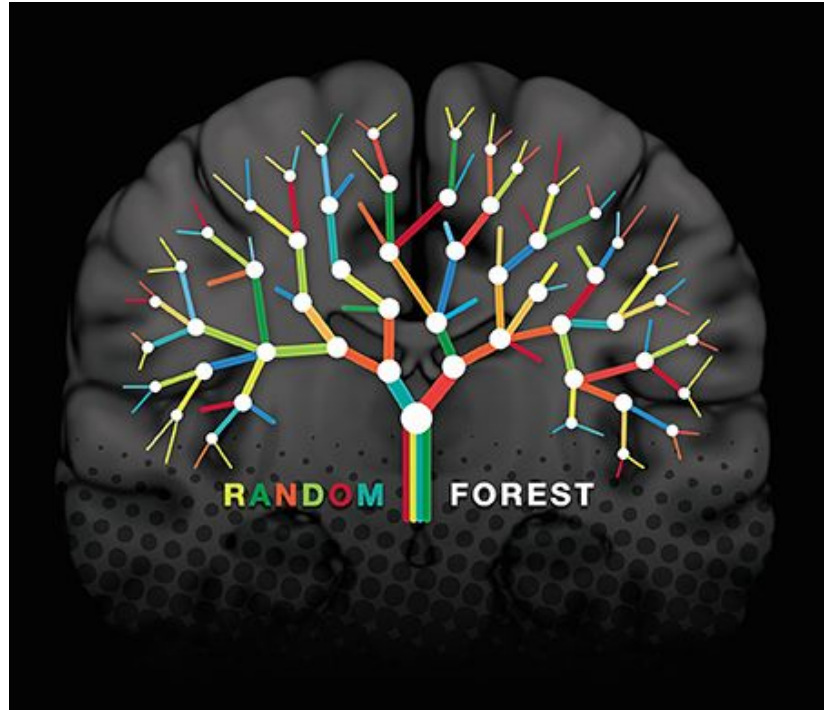
Can be used for regression and classification tasks



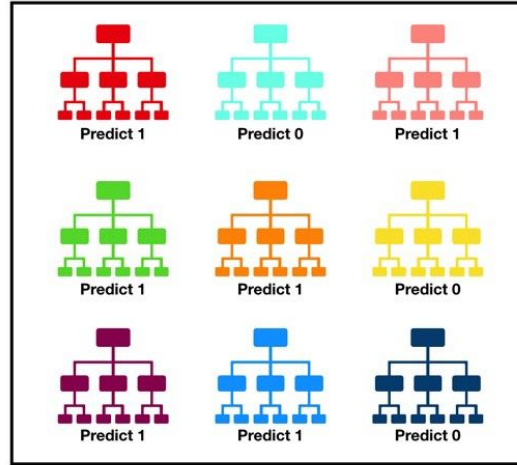
Why Forest?

Instead of a single decision, train multiple ones (10, 100, 1000...)

Use all the predictions of the trees to make final prediction (ensemble method)



Ensemble Method



Tally: Six 1s and Three 0s
Prediction: 1

Low risk that all models are making incorrect predictions at the same time => Performance

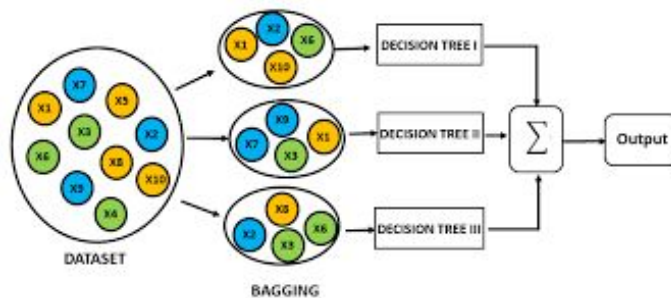
Can use simpler model (like decision tree) => Training Time

Why Random?

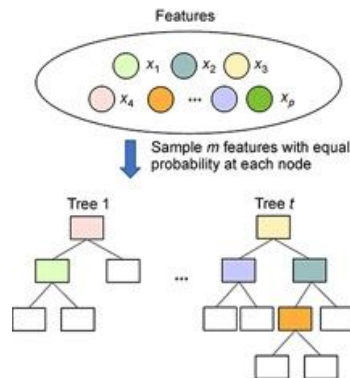
Random sampling of data
with replacement
(bootstrapping)

Random sampling of
feature

=> Uncorrelated trees



Random Forests

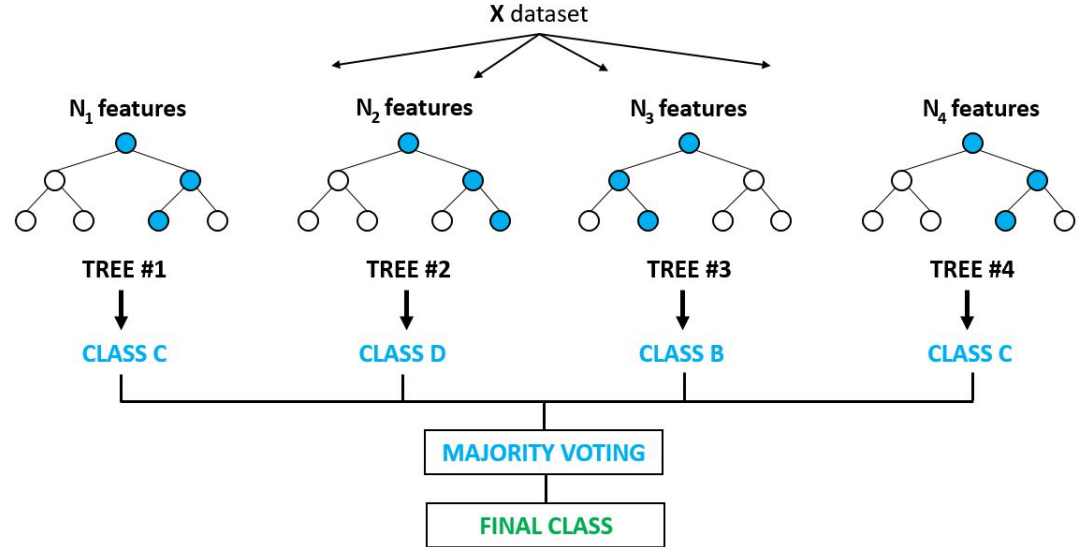


Hyperparameter: $n_estimator$

Number of trees to be trained

More trees, better performance

More trees, more computation

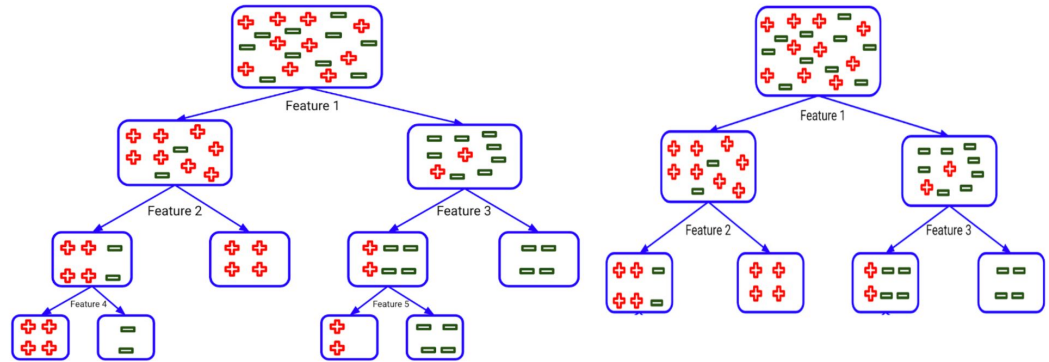


Hyperparameter: max_depth

Maximum number of nodes allowed

Deeper trees, better performance

Deeper trees, higher risk of overfitting

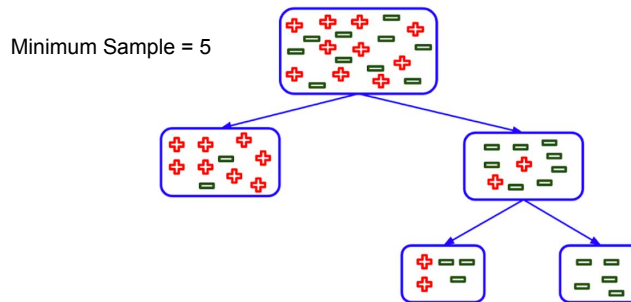
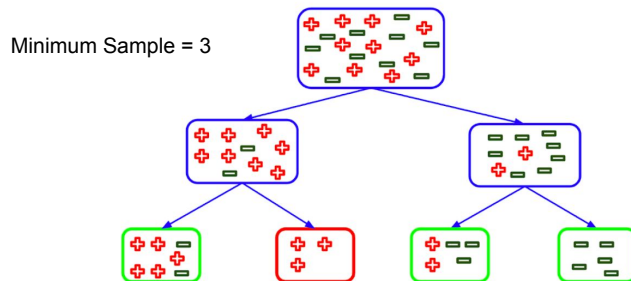


Hyperparameter: min_samples_leaf

Minimum number of observations in a leaf

Avoid very specific rules that apply to just few observations

Higher values, less risk of overfitting

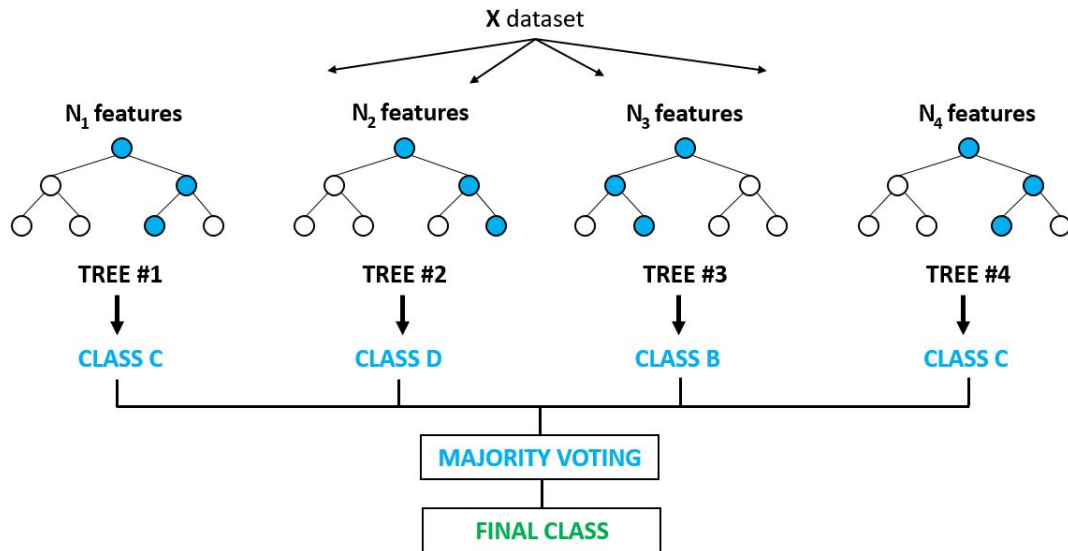


Hyperparameter: max_features

Maximum number of features to be used for each tree

More features, higher risk of overfitting

Less features, less performant



Scikit-learn RandomForest

Generic API for sklearn models:



Import class

```
from sklearn.ensemble import RandomForestClassifier
```



Instantiate a model with hyperparameter

```
rf = RandomForestClassifier(n_estimators=100, max_depth=10, min_samples_leaf=5,  
max_features=0.5)
```



Fit a model

```
rf.fit(X, y)
```



Predict with a model

```
rf.predict(new_X)
```

Sklearn Parameters

<https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html>

Parameters: **n_estimators** : *int*, **default=100**

The number of trees in the forest.

Changed in version 0.22: The default value of `n_estimators` changed from 10 to 100 in 0.22.

criterion : {*"gini"*, *"entropy"*}, **default="gini"**

The function to measure the quality of a split. Supported criteria are "gini" for the Gini impurity and "entropy" for the information gain. Note: this parameter is tree-specific.

max_depth : *int*, **default=None**

The maximum depth of the tree. If None, then nodes are expanded until all leaves are pure or until all leaves contain less than `min_samples_split` samples.

min_samples_split : *int or float*, **default=2**

The minimum number of samples required to split an internal node:

- If int, then consider `min_samples_split` as the minimum number.
- If float, then `min_samples_split` is a fraction and `ceil(min_samples_split * n_samples)` are the minimum number of samples for each split.

Changed in version 0.18: Added float values for fractions.

min_samples_leaf : *int or float*, **default=1**

The minimum number of samples required to be at a leaf node. A split point at any depth will only be considered if it leaves at least `min_samples_leaf` training samples in each of the left and right branches. This may have the effect of smoothing the model, especially in regression.

- If int, then consider `min_samples_leaf` as the minimum number.
- If float, then `min_samples_leaf` is a fraction and `ceil(min_samples_leaf * n_samples)` are the minimum number of samples for each node.

max_features : {*"auto"*, *"sqrt"*, *"log2"*}, *int or float*, **default="auto"**

The number of features to consider when looking for the best split:

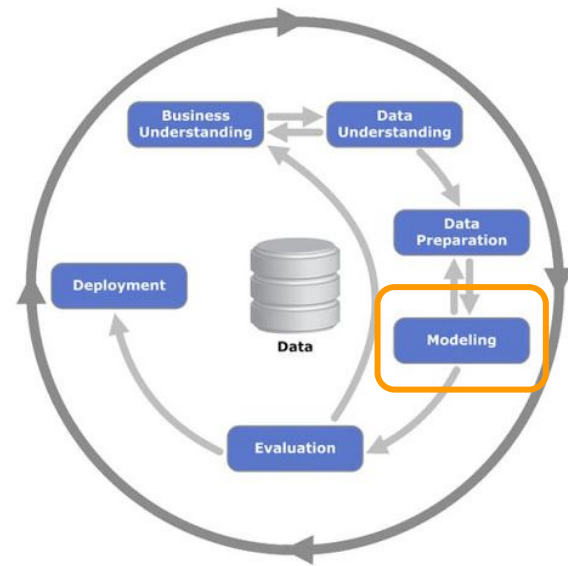
- If int, then consider `max_features` features at each split.
- If float, then `max_features` is a fraction and `int(max_features * n_features)` features are considered at each split.
- If "auto", then `max_features=sqrt(n_features)`.
- If "sqrt", then `max_features=sqrt(n_features)` (same as "auto").
- If "log2", then `max_features=log2(n_features)`.
- If None, then `max_features=n_features`.

Note: the search for a split does not stop until at least one valid partition of the node samples is found, even if it requires to effectively inspect more than `max_features` features.

max_leaf_nodes : *int*, **default=None**

Grow trees with `max_leaf_nodes` in best-first fashion. Best nodes are defined as relative reduction in impurity. If None then unlimited number of leaf nodes.

ExtraTrees



ExtraTrees

Extremely Randomized Trees

Very similar approach as for
RandomForest

Introduce randomness in the
combination features/data split

Single Split of Random Forest:

1. Select m features randomly
2. For each feature, **find the best cut-off point** using Gini/Entropy measure (combinations of features/data split)
3. Choose the best combination features/data split as the split node

Single Split of ExtraTrees:

1. Select m features randomly
2. For each feature, **select randomly a cut-off point** and calculate the Gini/Entropy measure
3. Choose the best combination features/data split as the split node

Difference between DT, RF and ET

	Decision Tree	Random Forest	Extra Trees
Number of trees	1	Many	Many
No of features considered for split at each decision node	All Features	Random subset of Features	Random subset of Features
Boostrapping(Drawing Sampling without replacement)	Not applied	Yes	No
How split is made	Best Split	Best Split	Random Split

Scikit-learn ExtraTrees

Generic API for sklearn models:



Import class

```
from sklearn.ensemble import ExtraTreesClassifier
```



Instantiate a model with hyperparameter

```
et = ExtraTreesClassifier(n_estimators=100, max_depth=10, min_samples_leaf=5,  
max_features=0.5)
```



Fit a model

```
et.fit(X, y)
```



Predict with a model

```
et.predict(new_X)
```

Sklearn Parameters

<https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.ExtraTreesClassifier.html?highlight=extratrees#sklearn.ensemble.ExtraTreesClassifier>

Parameters: `n_estimators : int, default=100`

The number of trees in the forest.

Changed in version 0.22: The default value of `n_estimators` changed from 10 to 100 in 0.22.

criterion : `{“gini”, “entropy”}, default=“gini”`

The function to measure the quality of a split. Supported criteria are “gini” for the Gini impurity and “entropy” for the information gain.

max_depth : `int, default=None`

The maximum depth of the tree. If None, then nodes are expanded until all leaves are pure or until all leaves contain less than `min_samples_split` samples.

min_samples_split : `int or float, default=2`

The minimum number of samples required to split an internal node:

- If int, then consider `min_samples_split` as the minimum number.
- If float, then `min_samples_split` is a fraction and `ceil(min_samples_split * n_samples)` are the minimum number of samples for each split.

Changed in version 0.18: Added float values for fractions.

min_samples_leaf : `int or float, default=1`

The minimum number of samples required to be at a leaf node. A split point at any depth will only be considered if it leaves at least `min_samples_leaf` training samples in each of the left and right branches. This may have the effect of smoothing the model, especially in regression.

- If int, then consider `min_samples_leaf` as the minimum number.
- If float, then `min_samples_leaf` is a fraction and `ceil(min_samples_leaf * n_samples)` are the minimum number of samples for each node.

Changed in version 0.18: Added float values for fractions.

max_features : `{“auto”, “sqrt”, “log2”}, int or float, default=“auto”`

The number of features to consider when looking for the best split:

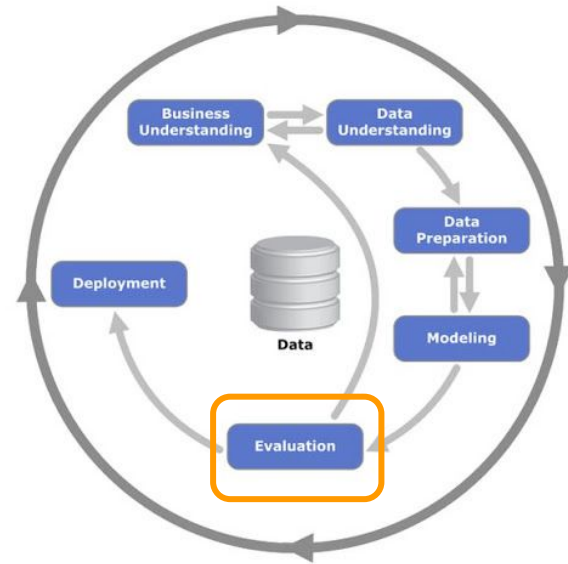
- If int, then consider `max_features` features at each split.
- If float, then `max_features` is a fraction and `int(max_features * n_features)` features are considered at each split.
- If “auto”, then `max_features=sqrt(n_features)`.
- If “sqrt”, then `max_features=sqrt(n_features)`.
- If “log2”, then `max_features=log2(n_features)`.
- If None, then `max_features=n_features`.

Note: the search for a split does not stop until at least one valid partition of the node samples is found, even if it requires to effectively inspect more than `max_features` features.

max_leaf_nodes : `int, default=None`

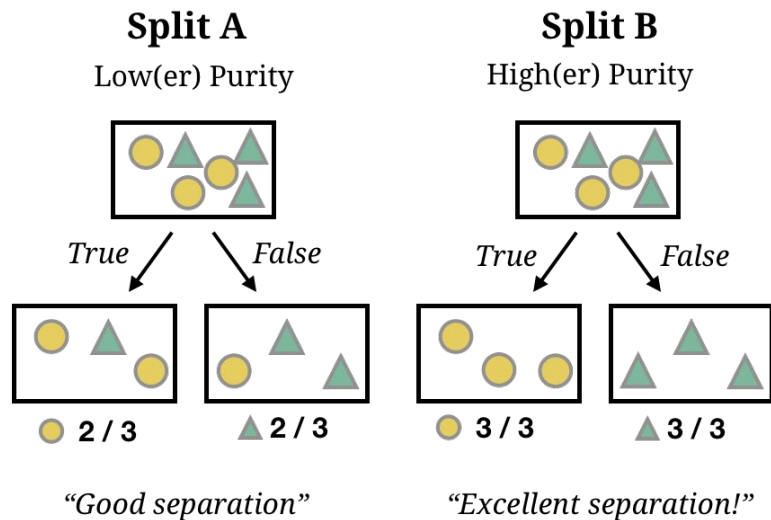
Grow trees with `max_leaf_nodes` in best-first fashion. Best nodes are defined as relative reduction in impurity. If None then unlimited number of leaf nodes.

Feature Importance



Impurity

RandomForest assess the level of impurity at each split



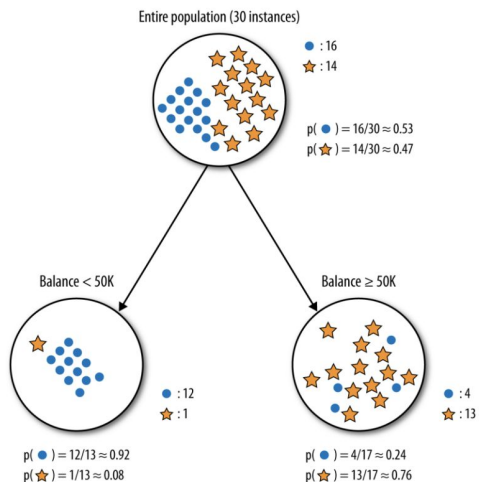
Entropy:

$$H(\text{distribution}) = \sum_{i=0}^n -p_i \log_i p_i$$

Gini:

$$\text{Gini}(\text{distribution}) = 1 - \sum_{i=0}^n p_i^2$$

Decision Tree Feature Importance



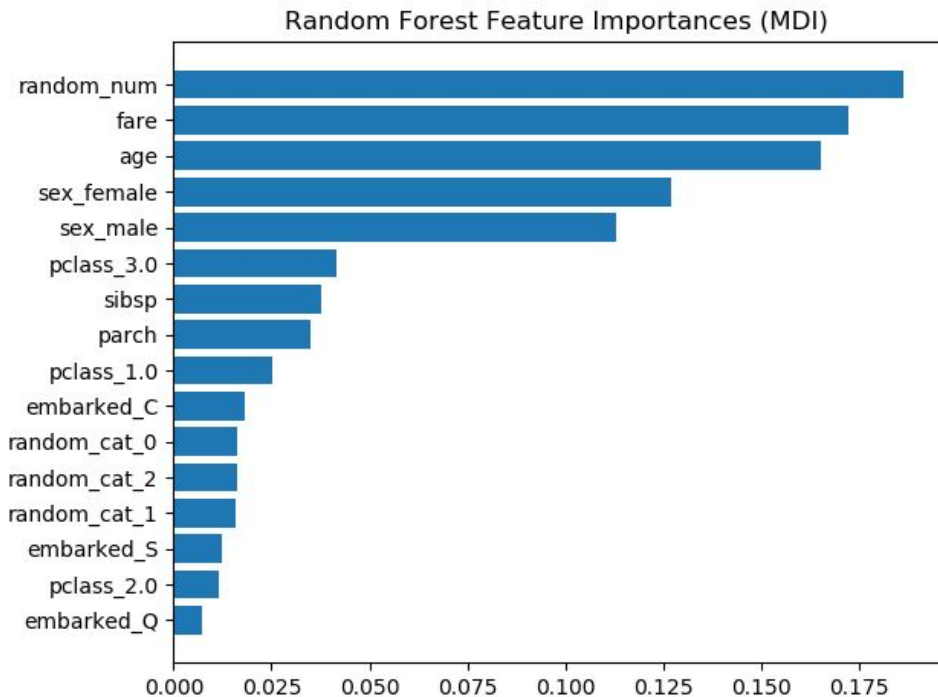
For each split of a tree, RandomForest calculates the change of impurity level for the given variable.

By summing these values for each variable, we know how much they contributed to the predictions

=> Feature Importance

$$\begin{aligned} IG &= \text{entropy}(\text{parent}) - [p(\text{Balance} < 50K) \times \text{entropy}(\text{Balance} < 50K) \\ &\quad + p(\text{Balance} \geq 50K) \times \text{entropy}(\text{Balance} \geq 50K)] \\ &\approx 0.99 - [0.43 \times 0.39 + 0.57 \times 0.79] \\ &\approx 0.37 \end{aligned}$$

RandomForest Feature Importance



For each tree, we know the feature importance of each variable.

Averaging them for all trees, gives the overall feature importance for RandomForest

Attributes:

base_estimator_ : *DecisionTreeClassifier*

The child estimator template used to create the collection of fitted sub-estimators.

estimators_ : *list of DecisionTreeClassifier*

The collection of fitted sub-estimators.

classes_ : *ndarray of shape (n_classes,)* or a list of such arrays

The classes labels (single output problem), or a list of arrays of class labels (multi-output problem).

n_classes_ : *int or list*

The number of classes (single output problem), or a list containing the number of classes for each output (multi-output problem).

n_features_ : *int*

The number of features when `fit` is performed.

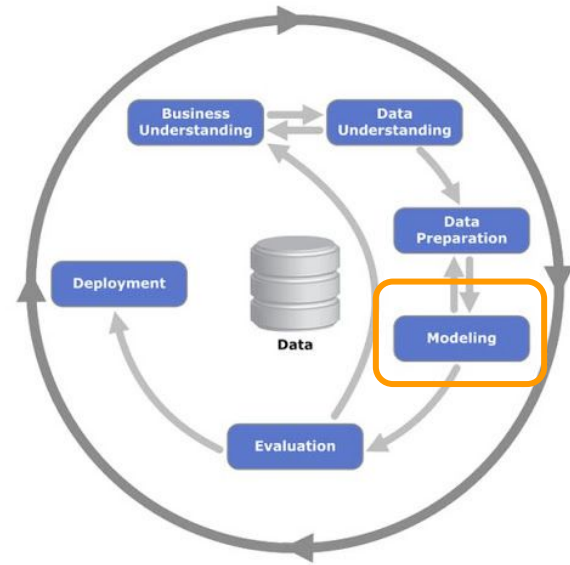
n_outputs_ : *int*

The number of outputs when `fit` is performed.

feature_importances_ : *ndarray of shape (n_features,)*

The impurity-based feature importances.

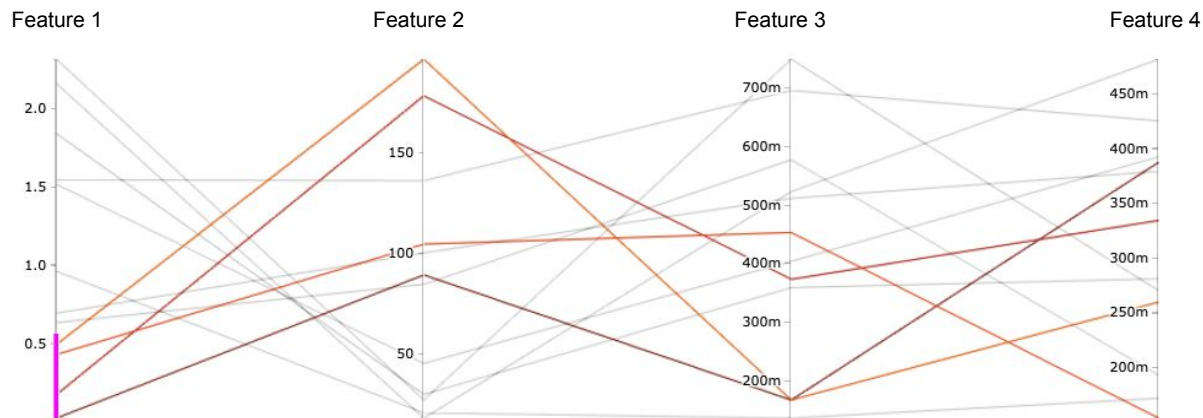
Hyperparameter Tuning



Hyperparameter Tuning

Hyperparameters can be continuous or discrete

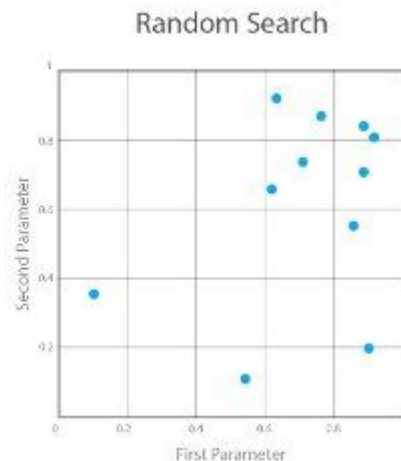
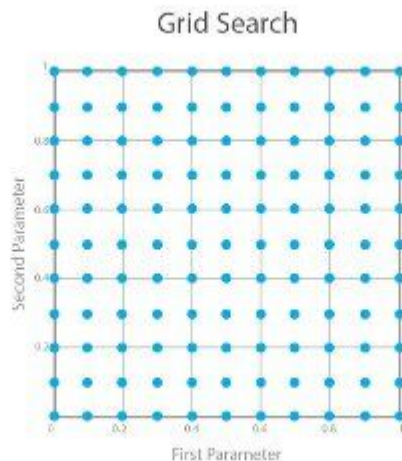
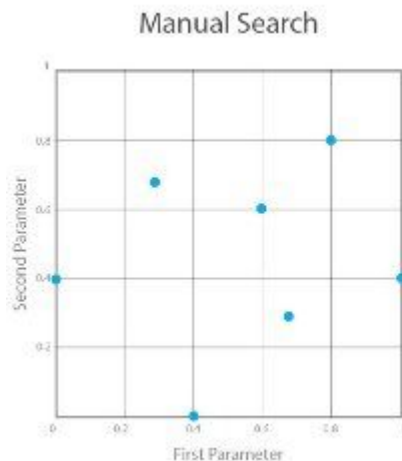
Even with just a few hyperparameters, the search space to find the best combination can be huge.



Automatic Hyperparameter Tuning

Hyperparameters Tuning can be very time-consuming.

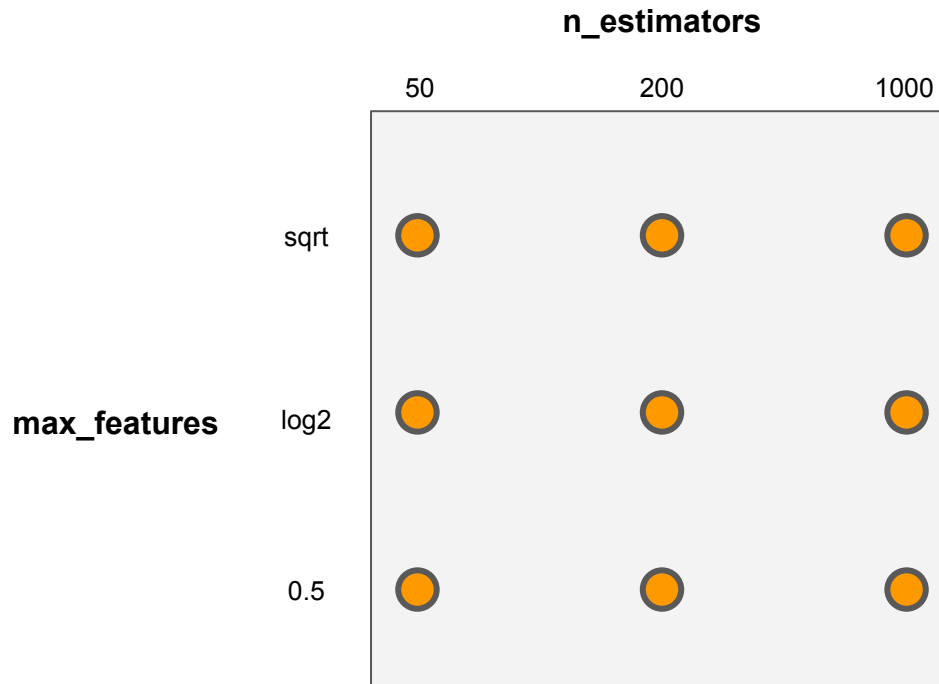
Needs to train a model for each combination of hyperparameters to be tuned.



Grid Search

Objective:

Build a matrix with all the possible combinations of hyperparameters and then train models on them to find the best one



Grid Search in sklearn

https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.GridSearchCV.html#sklearn.model_selection.GridSearchCV

```
from sklearn.ensemble import RandomForestClassifier

parameters = {'n_estimators':(50, 200, 1000), 'max_features':[0.3, 0.4, 0.5]}
rf = RandomForestClassifier()
```

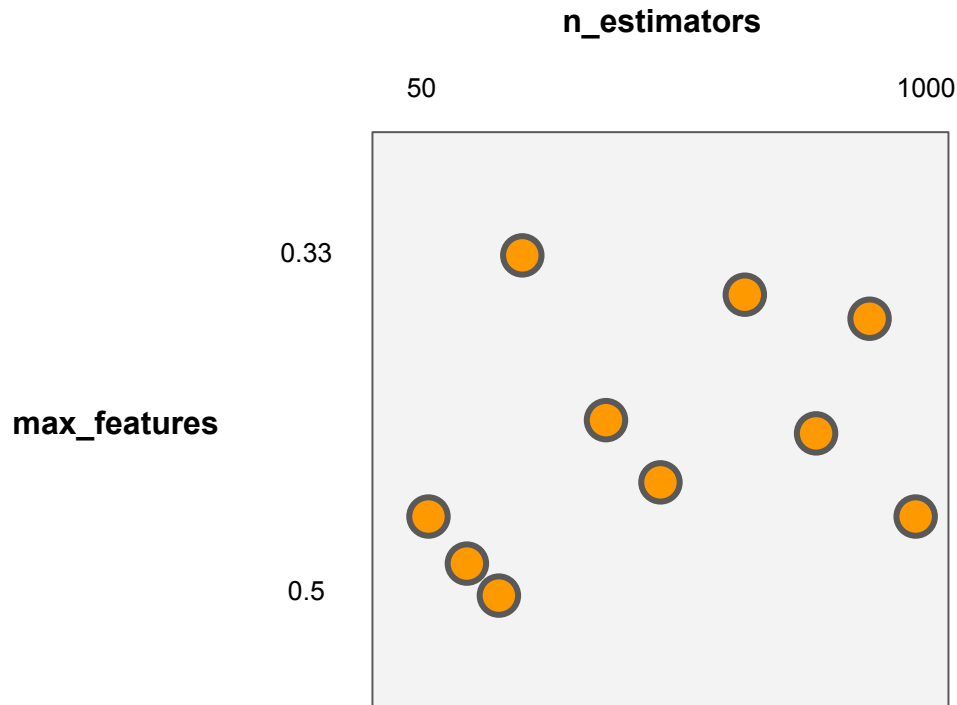
```
from sklearn.model_selection import GridSearchCV

search = GridSearchCV(rf, parameters)
search.fit(X_train, Y_train)
search.best_params_
```

Random Search

Objective:

Build a matrix with randomly sampled combinations of hyperparameters and then train models on them to find the best one



Random Search in sklearn

https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.RandomizedSearchCV.html#sklearn.model_selection.RandomizedSearchCV

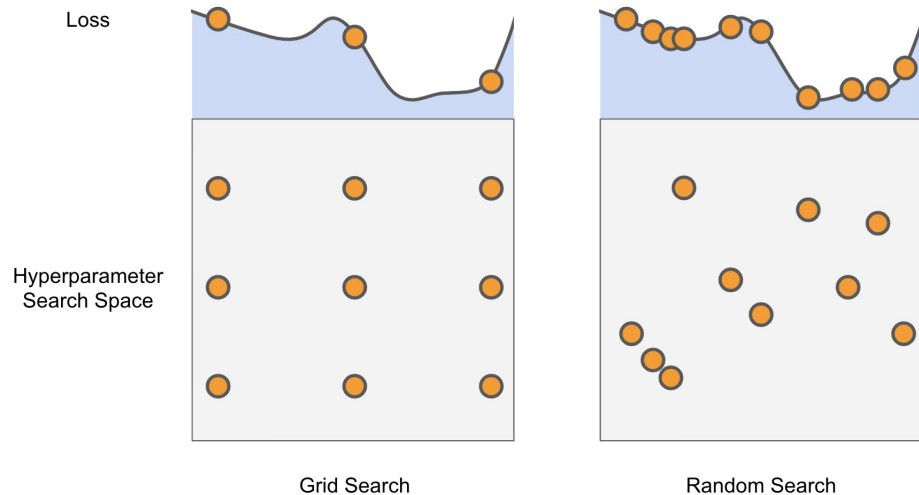
```
from sklearn.ensemble import RandomForestClassifier
from random import uniform

distributions = {
    'n_estimators': uniform(50, 1000),
    'max_features': uniform(0.3, 0.5)
}
rf = RandomForestClassifier()
```

```
from sklearn.model_selection import RandomizedSearchCV

search = RandomizedSearchCV(rf, distributions, random_state=0)
search.fit(X_train, Y_train)
search.best_params_
```

Grid versus Random Search



Grid Search explores exhaustively the search space.

Random Search has a better coverage of the search space.
More efficient



Time for Lab